BÀI THỰC HÀNH

# Ứng dụng quản lý nhân sự

Lập trình Android với Kotlin và SQLite

## BÁO CÁO KẾT QUẢ THỰC HIỆN

Sinh viên: ĐÔNG ĐỨC NGUYÊN
Mã sinh viên: B23DCVT311
Ngày thực hiện báo cáo: 23/10/2026

### MỤC TIÊU

Xây dựng ứng dụng lưu trữ và quản lý hồ sơ nhân viên trên thiết bị Android. Thực hiện tìm kiếm, thêm mới, chỉnh sửa, chọn ảnh và xóa có xác nhận.

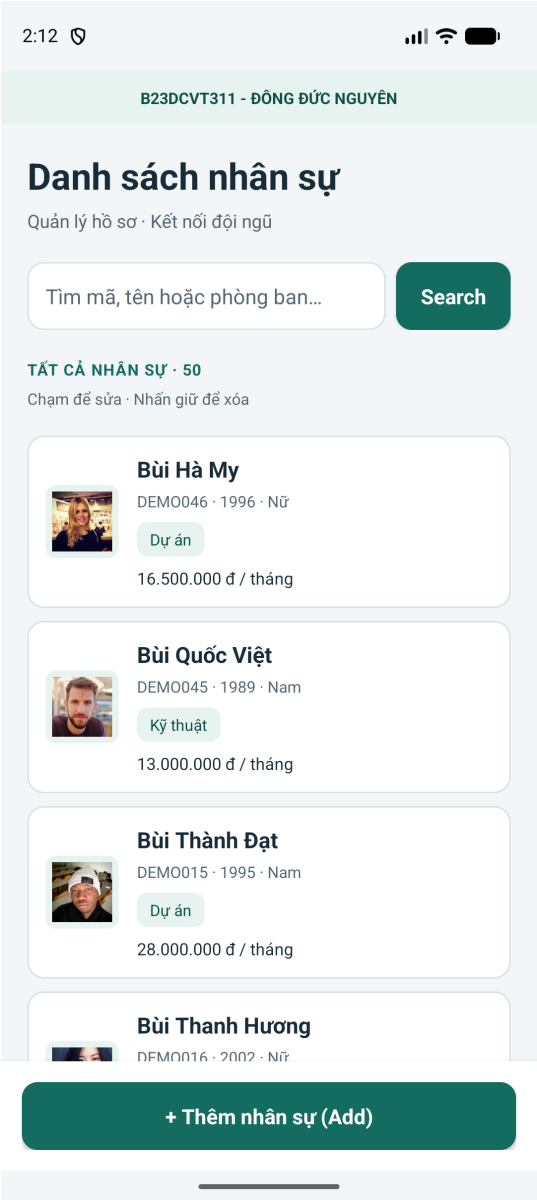
### PHẠM VI MINH CHỨNG

19 ảnh chụp màn hình thực tế trong một kịch bản thao tác có kiểm tra kết quả.

Dữ liệu ban đầu: 50 nhân sự mẫu, có ảnh chân dung và đủ bốn phòng ban.

### MÔI TRƯỜNG CHẠY

Máy ảo Pixel 9 / Android 17
Ứng dụng: vn.edu.nhansu
Cơ sở dữ liệu: nhansu.db




Hồ sơ mẫu là dữ liệu giả lập. Ảnh chân dung chỉ được dùng minh họa.

01 / TỔNG QUAN

# Yêu cầu và tổ chức chương trình

Ứng dụng có ba Activity, dùng chung lớp quản lý dữ liệu và form nhập liệu.

| Thành phần | Trách nhiệm |
|---|---|
| MainActivity | Hiển thị danh sách; tìm kiếm; mở Add/Detail; nhấn giữ để xác nhận xóa. |
| AddActivity | Nhập 9 trường. Add lưu hồ sơ; Refresh xóa nội dung của form. |
| DetailActivity | Hiển thị ảnh và thông tin có thể sửa. Update lưu; Cancel hủy thay đổi. |
| EmployeeFormActivity | Form chung, chọn ảnh, kiểm tra dữ liệu và xử lý nút lưu. |
| DBHelper / Employee | Tạo và truy vấn SQLite; biểu diễn đầy đủ thông tin một nhân sự. |
| EmployeeAdapter / PhotoLoader | Hiển thị từng dòng; đọc ảnh thu nhỏ ở luồng nền và lưu bộ nhớ đệm. |

## Luồng xử lý dữ liệu

**GIAO DIỆN**
Main / Add / Detail

**DBHELPER**
CRUD + tham số truy vấn

**SQLITE**
nhansu.db / employees

Môi trường phát triển: Kotlin 2.0.0, Android Gradle Plugin 8.7.3, Gradle 8.9, JDK 17; compile/target SDK 35, min SDK 26. Giao diện XML sử dụng View Binding.

**BỐ CỤC BÁO CÁO**

Trang 3: SQLite. Trang 4-8: trích mã nguồn chính. Trang 9-18: ảnh thao tác và kết quả. Trang 19: tổng hợp kiểm tra, nhận xét và nguồn tham khảo.

02 / CƠ SỞ DỮ LIỆU

# Thiết kế SQLite và xử lý nghiệp vụ

DBHelper kế thừa SQLiteOpenHelper; bảng employees lưu hồ sơ nhân sự.

| Cột | Kiểu | Ý nghĩa / ràng buộc |
|---|---|---|
| _id | INTEGER | Khóa chính tự tăng, ổn định khi sửa mã nhân sự. |
| code | TEXT | Mã nhân sự; NOT NULL, UNIQUE, COLLATE NOCASE. |
| full_name | TEXT | Họ và tên bắt buộc, không được rỗng. |
| birth_year | INTEGER | Năm sinh; form kiểm tra 1900 đến năm hiện tại. |
| gender | TEXT | Nam, Nữ hoặc Khác. |
| hometown / education | TEXT | Quê quán và trình độ, bắt buộc trên form. |
| photo_uri | TEXT | URI ảnh; có thể rỗng khi chưa chọn ảnh. |
| salary | INTEGER | Lương VNĐ không âm; ánh xạ Kotlin Long. |
| department | TEXT | Hành chính, Phần mềm, Kỹ thuật, Dự án. |
| search_text | TEXT | Chuỗi chuẩn hóa chữ thường và bỏ dấu để tìm kiếm. |

## Các phương thức chính

**getEmployees(keyword)** trả danh sách; **getEmployee(id)** lấy một hồ sơ.
**insertEmployee / updateEmployee / deleteEmployee** thực hiện CRUD.
**codeExists** kiểm tra trùng mã; UNIQUE bảo vệ thêm ở tầng SQLite.

**NGUYÊN TẮC LƯU DỮ LIỆU**

Chỉ Add/Update mới ghi hồ sơ. Cancel/Refresh không ghi thay đổi. Truy vấn dùng selectionArgs; cập nhật và xóa theo _id để tránh tác động nhầm người.

03 / MÃ NGUỒN

# Mô hình dữ liệu và tạo bảng

Employee biểu diễn một hồ sơ; DBHelper.onCreate tạo bảng employees khi mở nhansu.db lần đầu.

app/src/main/java/vn/edu/nhansu/data/Employee.kt · dòng 3–15

```kotlin
/** id là khóa nội bộ, mã nhân sự có thể được chỉnh sửa mà không đổi bản ghi. */
data class Employee(
    val id: Long = 0,
    val code: String,
    val fullName: String,
    val birthYear: Int,
    val gender: String,
    val hometown: String,
    val education: String,
    val photoUri: String = "",
    val salary: Long,
    val department: String,
)
```

Data class giữ đủ 9 thông tin của nhân sự cùng khóa nội bộ id. Lương dùng Long để lưu số nguyên VNĐ, tránh sai số của số thực.

app/src/main/java/vn/edu/nhansu/data/DBHelper.kt · dòng 9–31

```kotlin
/** Toàn bộ thao tác SQLite được đặt ở đây để Activity chỉ xử lý giao diện. */
class DBHelper(context: Context, databaseName: String = DATABASE_NAME) :
    SQLiteOpenHelper(context.applicationContext, databaseName, null, DATABASE_VERSION) {

    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL(
            """
            CREATE TABLE employees (
                _id INTEGER PRIMARY KEY AUTOINCREMENT,
                code TEXT NOT NULL COLLATE NOCASE UNIQUE CHECK(length(trim(code)) > 0),
                full_name TEXT NOT NULL CHECK(length(trim(full_name)) > 0),
                birth_year INTEGER NOT NULL CHECK(birth_year >= 1900),
                gender TEXT NOT NULL CHECK(gender IN ('Nam', 'Nữ', 'Khác')),
                hometown TEXT NOT NULL,
                education TEXT NOT NULL,
                photo_uri TEXT NOT NULL DEFAULT '',
                salary INTEGER NOT NULL CHECK(salary >= 0),
                department TEXT NOT NULL CHECK(department IN ('Hành chính', 'Phần mềm', 'Kỹ thuật', 'Dự án')),
                search_text TEXT NOT NULL
            )
            """.trimIndent()
        )
    }
```

Ràng buộc NOT NULL, UNIQUE và CHECK bảo vệ dữ liệu ở tầng SQLite, kể cả khi form đã kiểm tra trước. COLLATE NOCASE khiến mã “nv01” và “NV01” bị coi là trùng.

03 / MÃ NGUỒN

# Truy vấn và các thao tác CRUD

Mọi câu lệnh đều truyền giá trị qua selectionArgs/ContentValues, không nối chuỗi SQL.

app/src/main/java/vn/edu/nhansu/data/DBHelper.kt · dòng 39–66

```kotlin
fun getEmployees(keyword: String = ""): List<Employee> {
    val key = EmployeeRules.searchKey(keyword)
    // instr + selectionArgs coi %, _, dấu nháy là ký tự thường; không nối SQL.
    val selection = if (key.isEmpty()) null else "instr(search_text, ?) > 0"
    val args = if (key.isEmpty()) null else arrayOf(key)
    return readableDatabase.query(
        TABLE, null, selection, args, null, null, "full_name COLLATE LOCALIZED ASC, _id ASC"
    ).use { cursor -> buildList { while (cursor.moveToNext()) add(cursor.toEmployee()) } }
}

fun getEmployee(id: Long): Employee? = readableDatabase.query(
    TABLE, null, "_id = ?", arrayOf(id.toString()), null, null, null
).use { if (it.moveToFirst()) it.toEmployee() else null }

fun codeExists(code: String, excludedId: Long = 0): Boolean = readableDatabase.query(
    TABLE, arrayOf("_id"), "code = ? COLLATE NOCASE AND _id != ?",
    arrayOf(code.trim(), excludedId.toString()), null, null, null, "1"
).use { it.moveToFirst() }

fun insertEmployee(employee: Employee): Long =
    writableDatabase.insertOrThrow(TABLE, null, employee.toValues())

fun updateEmployee(employee: Employee): Boolean = writableDatabase.update(
    TABLE, employee.toValues(), "_id = ?", arrayOf(employee.id.toString())
) == 1

fun deleteEmployee(id: Long): Boolean =
    writableDatabase.delete(TABLE, "_id = ?", arrayOf(id.toString())) == 1
```

getEmployees chuẩn hóa từ khóa rồi tìm trong cột search_text bằng instr, nên ký tự %, _ hoặc dấu nháy được coi là chữ bình thường. Sửa và xóa theo _id; kết quả == 1 cho biết đúng một bản ghi bị tác động.

app/src/main/java/vn/edu/nhansu/data/DBHelper.kt · dòng 68–79

```kotlin
private fun Employee.toValues() = ContentValues().apply {
    put("code", code.trim())
    put("full_name", fullName.trim())
    put("birth_year", birthYear)
    put("gender", gender)
    put("hometown", hometown.trim())
    put("education", education.trim())
    put("photo_uri", photoUri)
    put("salary", salary)
    put("department", department)
    put("search_text", EmployeeRules.searchKey("$code $fullName $hometown $education $department"))
}
```

Trước khi ghi, dữ liệu được cắt khoảng trắng; search_text được tính lại từ mã, tên, quê quán, trình độ và phòng ban.

03 / MÃ NGUỒN

# Chuẩn hóa tìm kiếm và kiểm tra dữ liệu

EmployeeRules chứa quy tắc thuần Kotlin, có unit test riêng; form dùng các quy tắc này trước khi lưu.

app/src/main/java/vn/edu/nhansu/data/EmployeeRules.kt · dòng 7–23

```kotlin
object EmployeeRules {
    val genders = listOf("Nam", "Nữ", "Khác")
    val departments = listOf("Hành chính", "Phần mềm", "Kỹ thuật", "Dự án")

    /** Chuẩn hóa để tìm kiếm cả tiếng Việt có dấu và không dấu. */
    fun searchKey(value: String): String = Normalizer.normalize(value, Normalizer.Form.NFD)
        .replace(Regex("\\p{M}+"), "")
        .replace('đ', 'd').replace('Đ', 'D')
        .lowercase(Locale.ROOT).trim().replace(Regex("\\s+"), " ")

    fun validBirthYear(value: Int): Boolean = value in 1900..Year.now().value

    // Lương nhập bằng số nguyên VNĐ, tránh sai số của Float/Double.
    fun parseSalary(value: String): Long? = value.trim()
        .takeIf { it.matches(Regex("[0-9]+")) }
        ?.toLongOrNull()?.takeIf { it >= 0 }
}
```

searchKey tách dấu theo chuẩn NFD, bỏ dấu thanh, đổi đ → d và gộp khoảng trắng, nhờ đó “phan mem” tìm được “Phần mềm”. parseSalary chỉ nhận chuỗi chữ số và trả null nếu tràn số.

app/src/main/java/vn/edu/nhansu/ui/EmployeeFormActivity.kt · dòng 114–122, 135–146

```kotlin
private fun readEmployee(): Employee? = with(binding) {
    val requiredFields = listOf(codeInput, nameInput, yearInput, hometownInput, educationInput, salaryInput)
    requiredFields.forEach { it.error = null }
    val emptyField = requiredFields.firstOrNull { it.text.isNullOrBlank() }
    if (emptyField != null) return invalid(emptyField, R.string.required_error)
    val year = yearInput.text.toString().toIntOrNull()
    if (year == null || !EmployeeRules.validBirthYear(year)) return invalid(yearInput, R.string.year_error)
    val salary = EmployeeRules.parseSalary(salaryInput.text.toString())
        ?: return invalid(salaryInput, R.string.salary_error)
    // ...
    Employee(
        id = employeeId,
        code = codeInput.text.toString().trim(),
        fullName = nameInput.text.toString().trim(),
        birthYear = year,
        gender = genderSpinner.selectedItem.toString(),
        hometown = hometownInput.text.toString().trim(),
        education = educationInput.text.toString().trim(),
        photoUri = selectedPhotoUri,
        salary = salary,
        department = departmentSpinner.selectedItem.toString(),
    )
```

readEmployee báo lỗi ngay tại ô đầu tiên còn trống hoặc sai định dạng (Hình 4), sau đó kiểm tra năm sinh, lương, giới tính và phòng ban. Chỉ khi tất cả hợp lệ mới tạo đối tượng Employee.

03 / MÃ NGUỒN

# Form dùng chung: Add, Refresh, Cancel, Update

AddActivity và DetailActivity là hai Activity riêng, chỉ khác giá trị isEditing.

app/src/main/java/vn/edu/nhansu/ui/AddActivity.kt · dòng 3–6

```kotlin
/** Refresh xóa form; Add ghi SQLite rồi trả RESULT_OK về Main. */
class AddActivity : EmployeeFormActivity() {
    override val isEditing = false
}
```

DetailActivity khai báo tương tự với isEditing = true và hằng EXTRA_EMPLOYEE_ID để nhận _id từ Main.

app/src/main/java/vn/edu/nhansu/ui/EmployeeFormActivity.kt · dòng 77–80, 155–181

```kotlin
    binding.secondaryButton.setOnClickListener {
        if (isEditing) finish() else refreshForm()
    }
    binding.saveButton.setOnClickListener { saveEmployee() }
// ...
private fun saveEmployee() {
    if (saving) return
    val employee = readEmployee() ?: return
    saving = true
    binding.saveButton.isEnabled = false
    try {
        if (db.codeExists(employee.code, employeeId)) {
            invalid(binding.codeInput, R.string.duplicate_code)
            return
        }
        if (isEditing) {
            if (!db.updateEmployee(employee)) { toast(R.string.employee_missing); return }
        } else {
            db.insertEmployee(employee)
        }
        toast(if (isEditing) R.string.updated else R.string.added)
        setResult(RESULT_OK)
        finish() // Trở lại Main đang có sẵn, không tạo thêm một MainActivity.
    } catch (_: SQLiteConstraintException) {
        invalid(binding.codeInput, R.string.duplicate_code)
    } catch (_: SQLiteException) {
        toast(R.string.database_error)
    } finally {
        saving = false
        binding.saveButton.isEnabled = true
    }
}
```

Nút bên trái là Refresh (Add) hoặc Cancel (Detail), không gọi DBHelper. Nút bên phải kiểm tra mã trùng rồi insert/update; biến saving chặn nhấn hai lần. Thành công thì trả RESULT_OK để Main tải lại danh sách.

03 / MÃ NGUỒN

# Màn hình Main: mở form, xóa và chọn ảnh

Main nhận kết quả từ Add/Detail; nhấn giữ một dòng để xác nhận xóa.

app/src/main/java/vn/edu/nhansu/ui/MainActivity.kt · dòng 20–27, 49–57, 87–100

```
20  private val editEmployee = registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { result
        ->
21      if (result.resultCode == RESULT_OK) {
22          // Sau Add/Update hiển thị toàn bộ, kể cả hồ sơ vừa đổi tên/phòng ban.
23          activeQuery = ""
24          binding.searchInput.text.clear()
25          loadEmployees()
26      }
27  }
        // ...
49      binding.employeeList.setOnItemClickListener { _, _, position, _ ->
50          editEmployee.launch(Intent(this, DetailActivity::class.java).putExtra(
51              DetailActivity.EXTRA_EMPLOYEE_ID, employeeAdapter.getItem(position).id
52          ))
53      }
54      binding.employeeList.setOnItemLongClickListener { _, _, position, _ ->
55          confirmDelete(employeeAdapter.getItem(position))
56          true // Không phát sinh thêm sự kiện click mở Detail.
57      }
    // ...
87  private fun confirmDelete(employee: Employee) {
88      AlertDialog.Builder(this)
89          .setTitle(R.string.delete_title)
90          .setMessage(getString(R.string.delete_message, employee.fullName, employee.code))
91          .setNegativeButton(R.string.no, null)
92          .setPositiveButton(R.string.yes) { _, _ ->
93              try {
94                  toast(if (db.deleteEmployee(employee.id)) R.string.deleted else R.string.employee_missing)
95                  loadEmployees()
96              } catch (_: SQLiteException) {
97                  toast(R.string.database_error)
98              }
99          }.show()
100 }
```

Add/Update thành công thì xóa bộ lọc và tải lại danh sách. Chạm một dòng mở Detail kèm _id; nhấn giữ trả về true để không mở nhầm Detail. Chỉ nút Có (Yes) mới gọi deleteEmployee; Không (No) truyền listener null nên chỉ đóng hộp thoại.

app/src/main/java/vn/edu/nhansu/ui/EmployeeFormActivity.kt · dòng 25–36

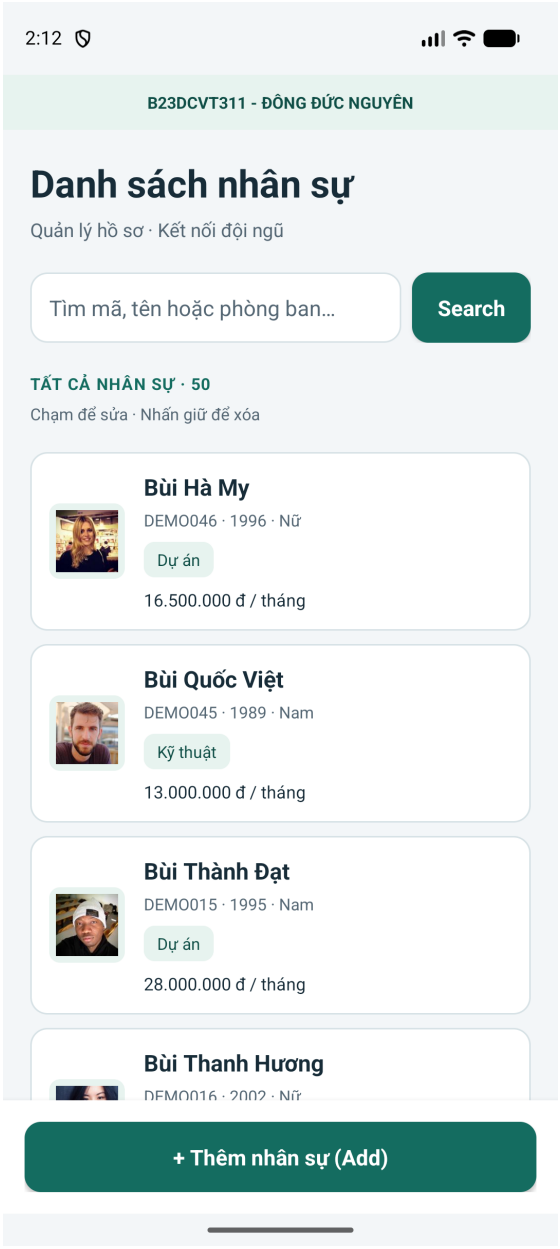
```
25  private val choosePhoto = registerForActivityResult(ActivityResultContracts.OpenDocument()) { uri ->
26      if (uri != null) {
27          try {
28              // Giữ quyền đọc ảnh sau khi đóng/mở ứng dụng, không cần quyền toàn bộ thư viện.
29              contentResolver.takePersistableUriPermission(uri, Intent.FLAG_GRANT_READ_URI_PERMISSION)
30              selectedPhotoUri = uri.toString()
31              showPhoto()
32          } catch (_: Exception) {
33              toast(R.string.photo_error)
34          }
35      }
36  }
```

Ảnh chọn bằng OpenDocument; takePersistableUriPermission giữ quyền đọc sau khi mở lại ứng dụng. SQLite chỉ lưu URI, không lưu ảnh dạng BLOB.

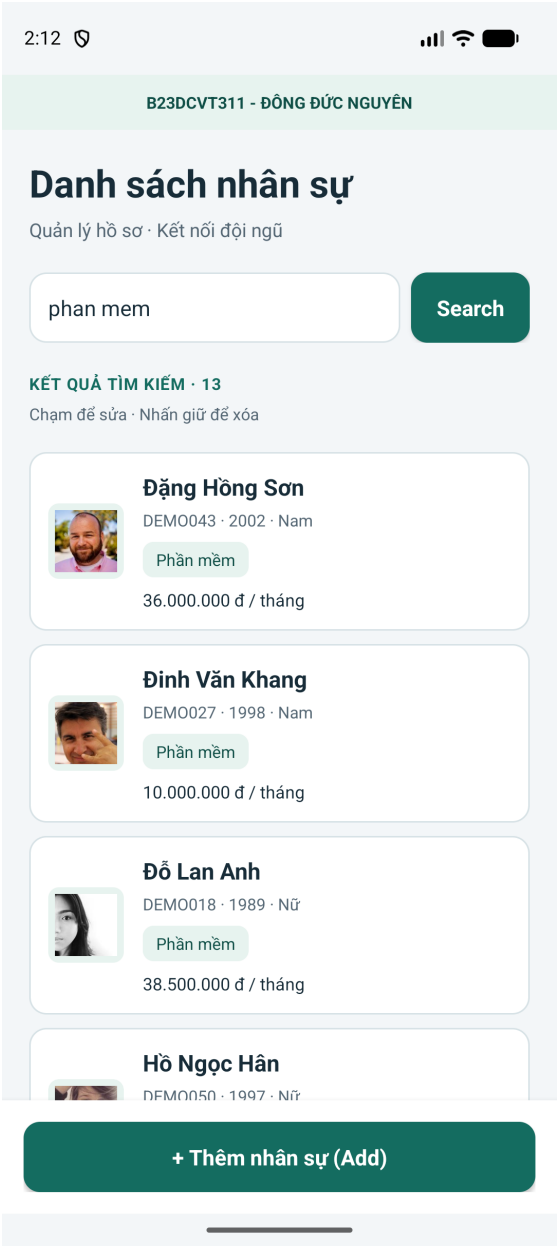
04 / DANH SÁCH VÀ TÌM KIẾM

# Hiển thị danh sách và Search

Mở Main; nhập “phan mem” không dấu rồi nhấn Search để lọc phòng ban Phần mềm.




Hình 1. Danh sách ban đầu có 50 nhân sự.




Hình 2. Tìm kiếm không dấu theo phòng ban.

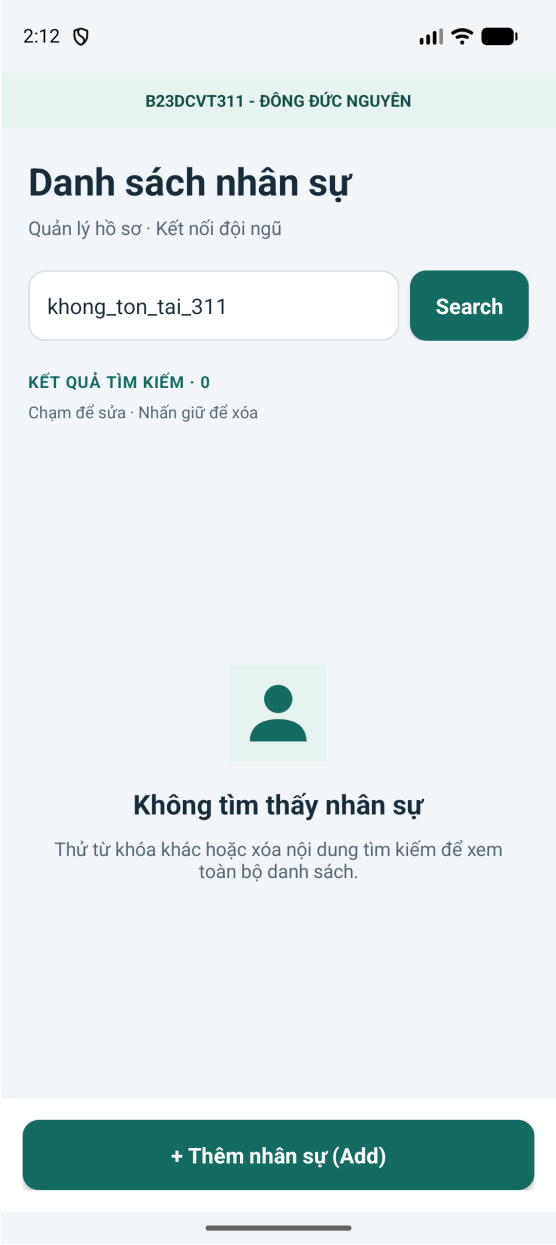
## KẾT QUẢ QUAN SÁT

Danh sách hiển thị ảnh, tên, mã, năm sinh, giới tính, phòng ban và lương. Từ khóa không dấu vẫn trả về nhân sự thuộc phòng Phần mềm.

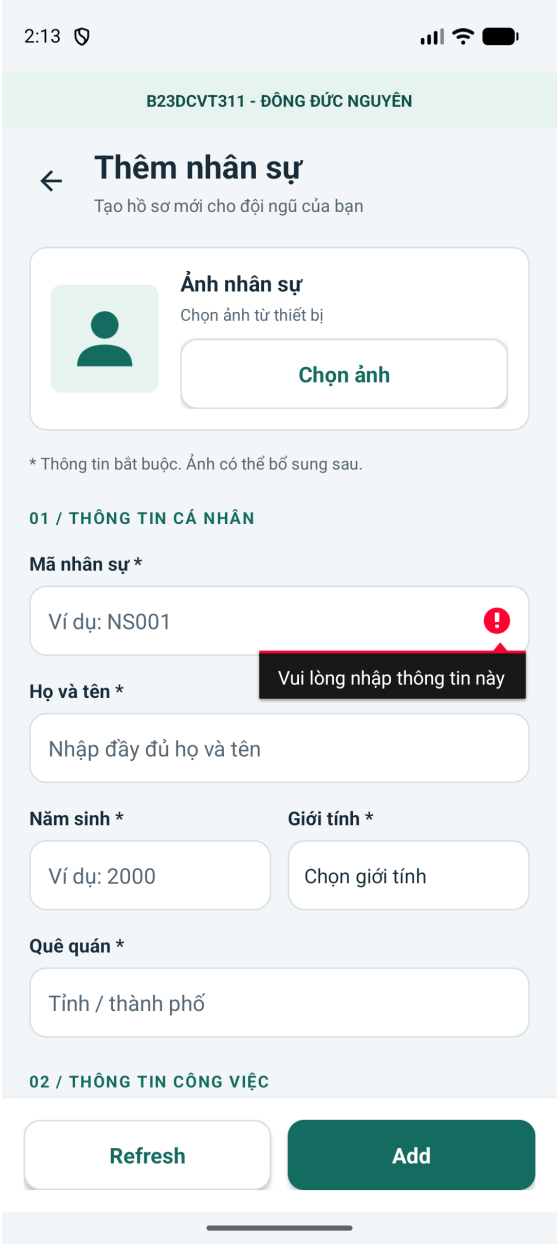
05 / TÌNH HUỐNG BIÊN

# Không có kết quả và kiểm tra bắt buộc

Thử từ khóa không tồn tại; tại Add, nhấn Add khi chưa nhập thông tin.




Hình 3. Thông báo khi không có kết quả tìm kiếm.




Hình 4. Báo lỗi tại trường Mã nhân sự còn trống.

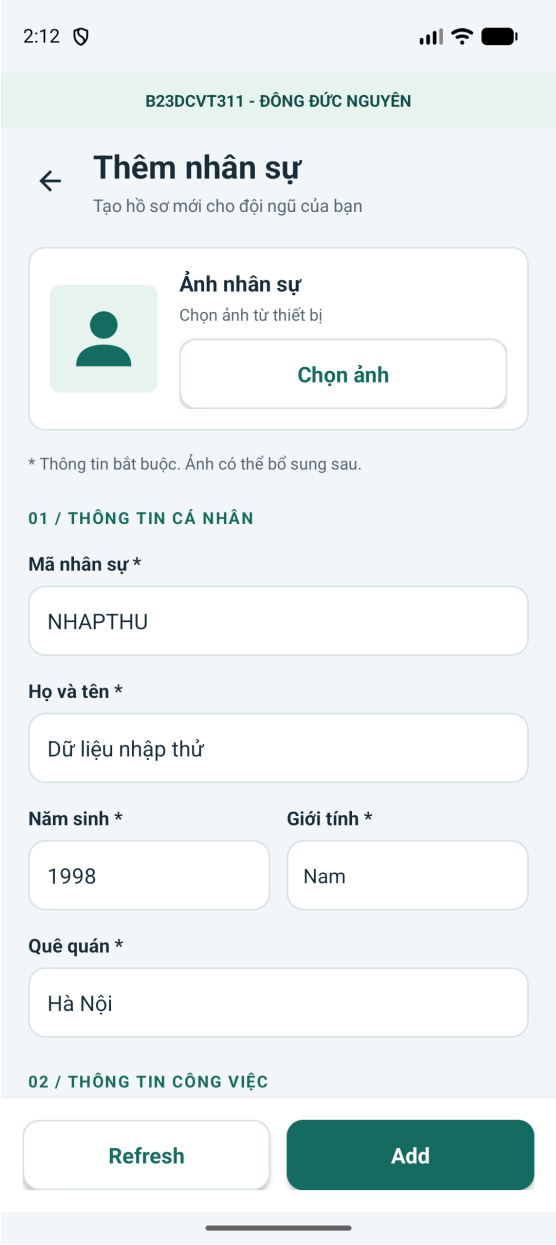
## KẾT QUẢ QUAN SÁT

Ứng dụng có trạng thái danh sách rỗng rõ ràng. Form chặn lưu thiếu dữ liệu và chỉ ra trường cần nhập; chưa phát sinh nhân sự mới.

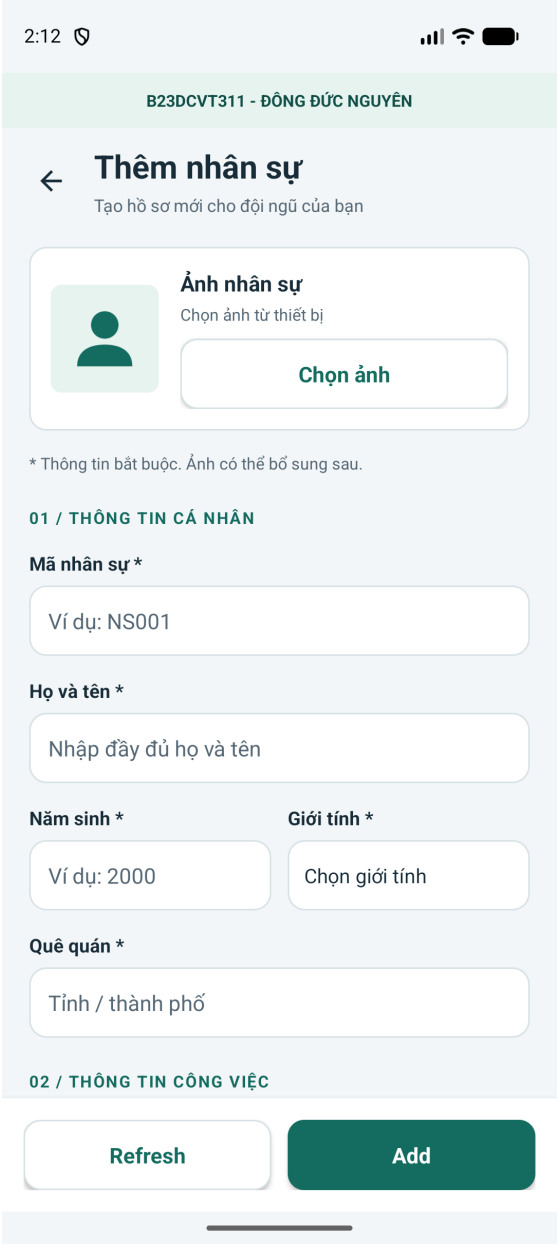
06 / LÀM MỚI FORM

# Refresh xóa nội dung đã nhập

Nhập dữ liệu thử vào form và chọn giới tính, phòng ban; sau đó nhấn Refresh.




Hình 5. Form đang chứa dữ liệu nhập thử.




Hình 6. Form sau thao tác Refresh.

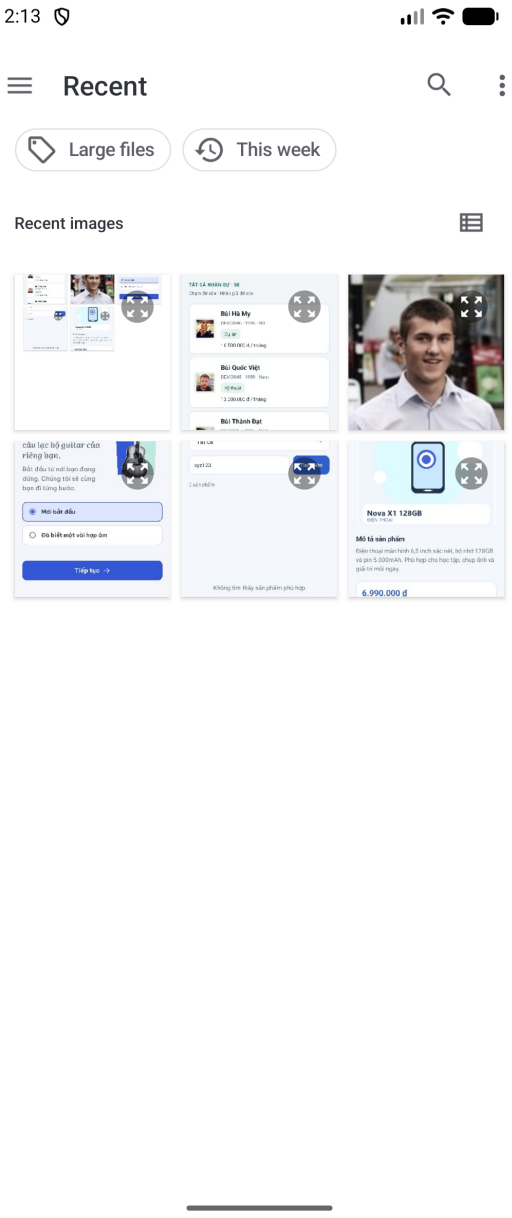
## KẾT QUẢ QUAN SÁT

Các ô nhập trở về rỗng, lựa chọn trở về trạng thái ban đầu. Kịch bản kiểm tra các ô văn bản đã được xóa; thao tác này không thêm bản ghi vào SQLite.

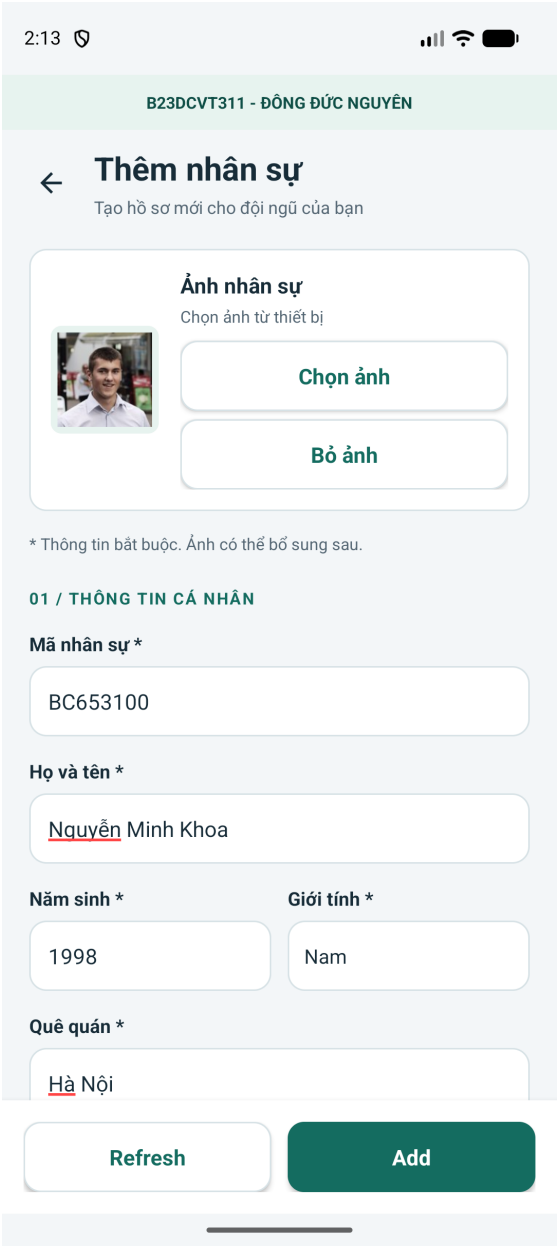
07 / ẢNH VÀ THÔNG TIN CÁ NHÂN

# Chọn ảnh từ thiết bị

Nhấn Chọn ảnh, chọn bao-cao-avatar.jpg trong trình chọn tệp, rồi nhập thông tin cá nhân.




Hình 7. Trình chọn ảnh của hệ thống Android.




Hình 8. Ảnh đã chọn và thông tin cá nhân trên form.

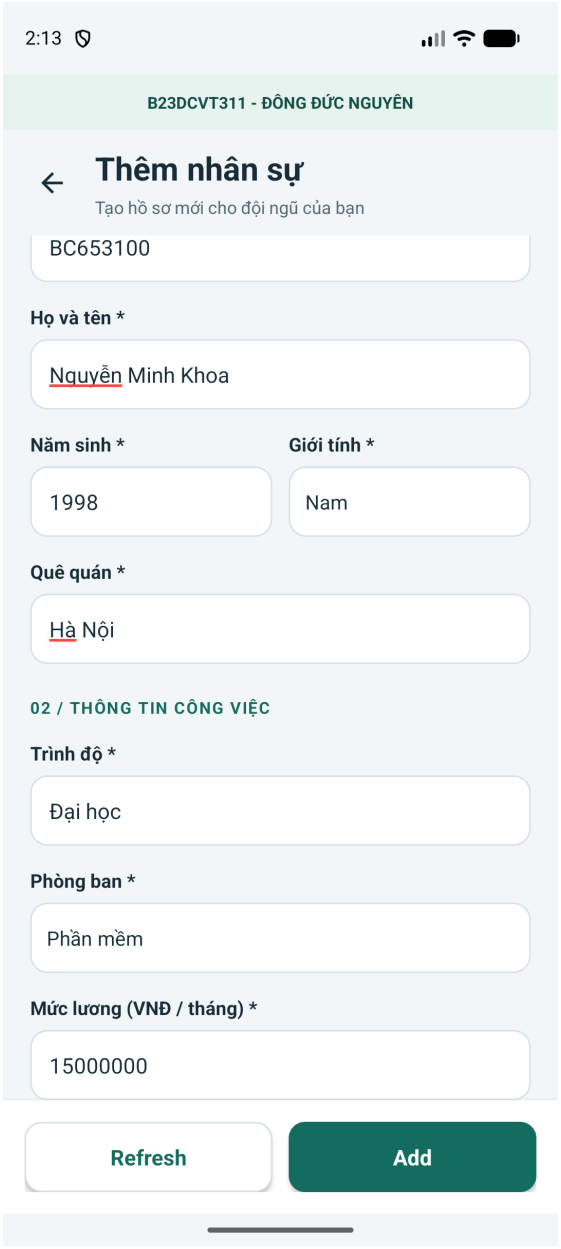
**KẾT QUẢ QUAN SÁT**

Ảnh được hiển thị xem trước. Hồ sơ dùng tên giả lập Nguyễn Minh Khoa; URI ảnh được lưu và kiểm tra đọc thành công sau khi thêm.

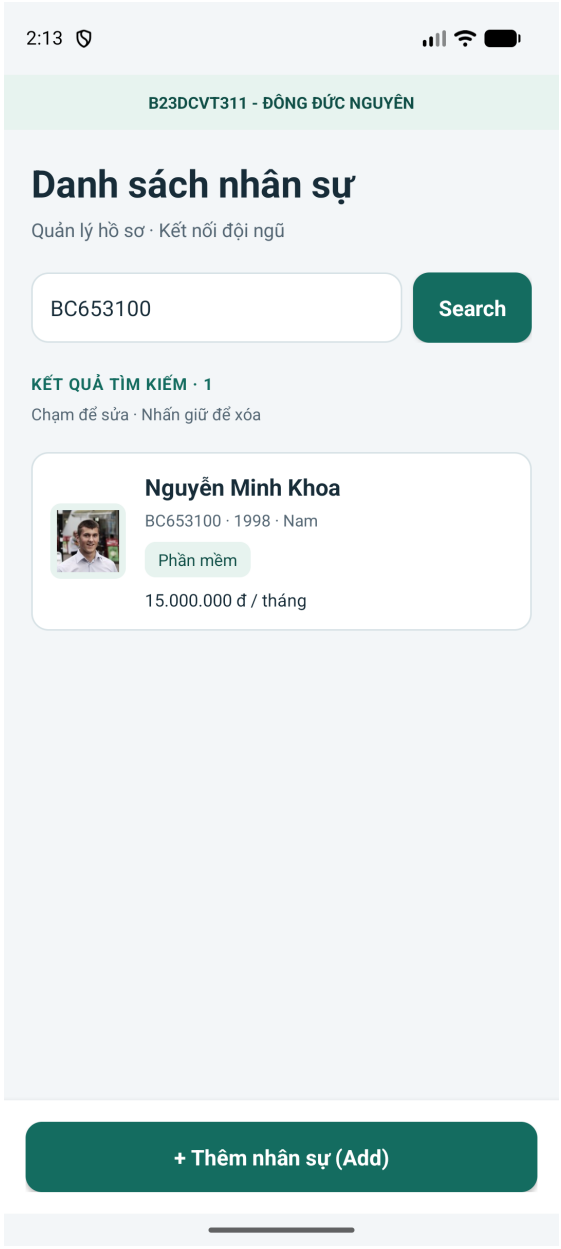
08 / THÊM NHÂN SỰ

# Add lưu hồ sơ và cập nhật Main

Nhập trình độ Đại học, phòng Phần mềm và lương 15.000.000 VNĐ; nhấn Add.




Hình 9. Phần thông tin công việc trước khi lưu.




Hình 10. Hồ sơ vừa thêm được tìm theo mã riêng.

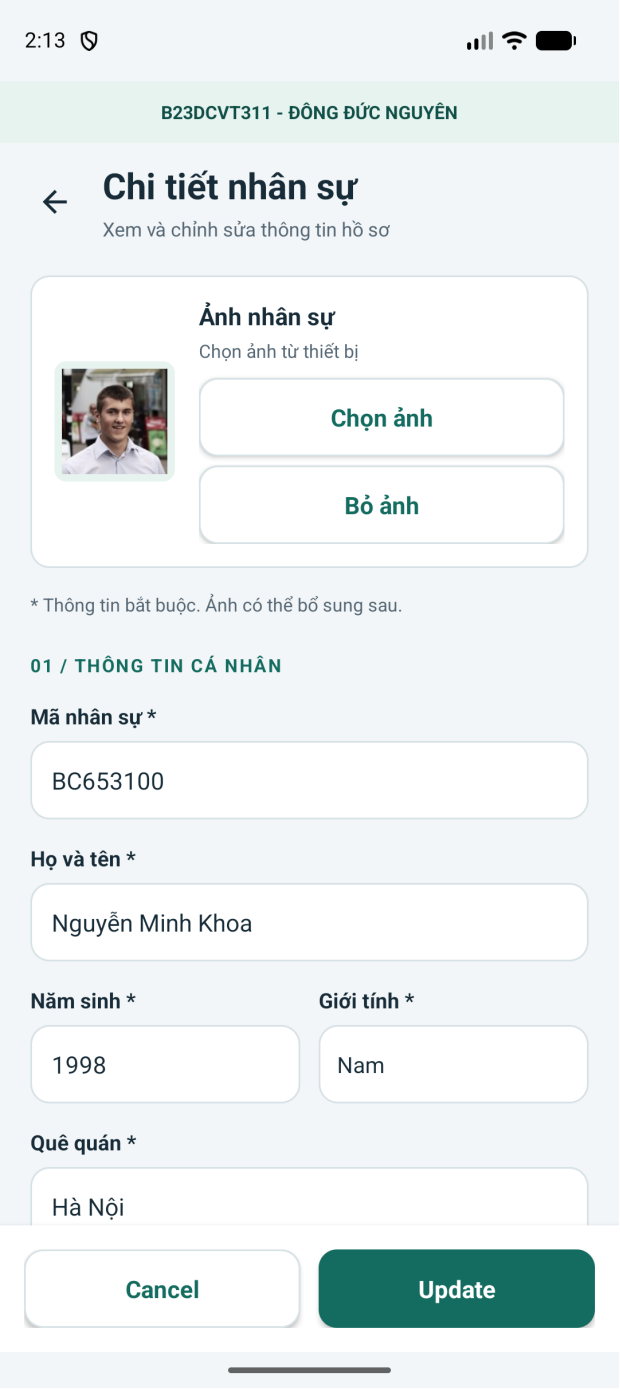
## KẾT QUẢ QUAN SÁT

Add thành công, số bản ghi tăng từ 50 lên 51. Main được cập nhật; tìm theo mã hồ sơ tạm trả về đúng một nhân sự có ảnh và lương đã nhập.

09 / CHI TIẾT NHÂN SỰ

# Mở hồ sơ từ một dòng danh sách

Chạm vào hồ sơ vừa tạo để mở DetailActivity.




## Dữ liệu được nạp sẵn

Ảnh, mã nhân sự, họ tên, năm sinh, giới tính và các thông tin công việc được điền từ SQLite theo _id.

## Có thể chỉnh sửa

Các ô trên Detail vẫn cho nhập lại. Nội dung sửa chỉ nằm trên form cho đến khi nhấn Update.

## Hai nút ở cuối

**Cancel:** đóng màn hình và giữ dữ liệu cũ.

**Update:** kiểm tra dữ liệu, cập nhật hồ sơ và quay về Main.

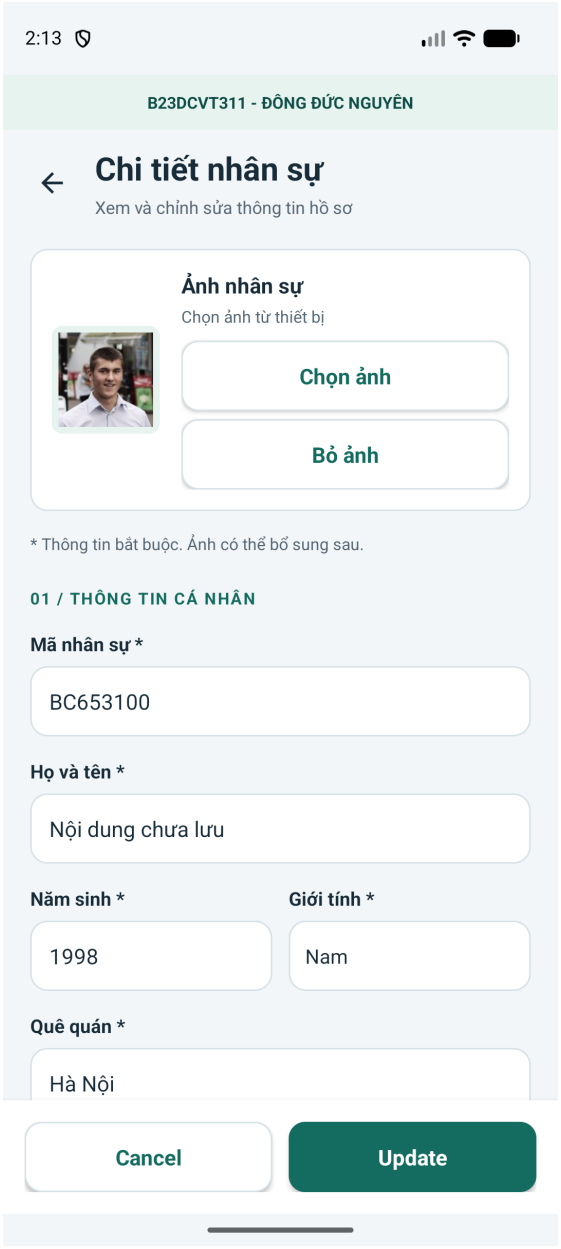
**Hình 11. Chi tiết hồ sơ với ảnh và thông tin đã lưu.**

Form có thể cuộn; nút Cancel/Update đặt ngoài vùng cuộn để luôn dễ thao tác.

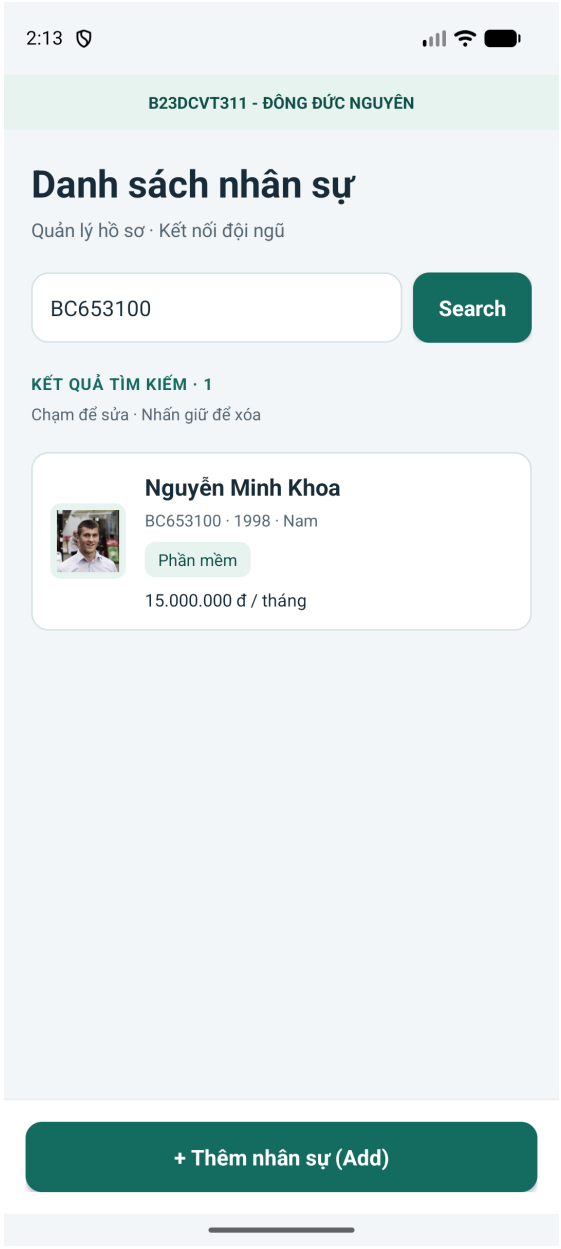
10 / HỦY CHỈNH SỬA

# Cancel không cập nhật dữ liệu

Đổi họ tên thành “Nội dung chưa lưu”, sau đó nhấn Cancel để trở về Main.




Hình 12. Nội dung thay đổi đang nằm trên form.




Hình 13. Sau Cancel, họ tên cũ vẫn được giữ lại.

## KẾT QUẢ QUAN SÁT

Kiểm tra SQLite xác nhận họ tên vẫn là Nguyễn Minh Khoa. Không có thao tác update khi người dùng nhấn Cancel.

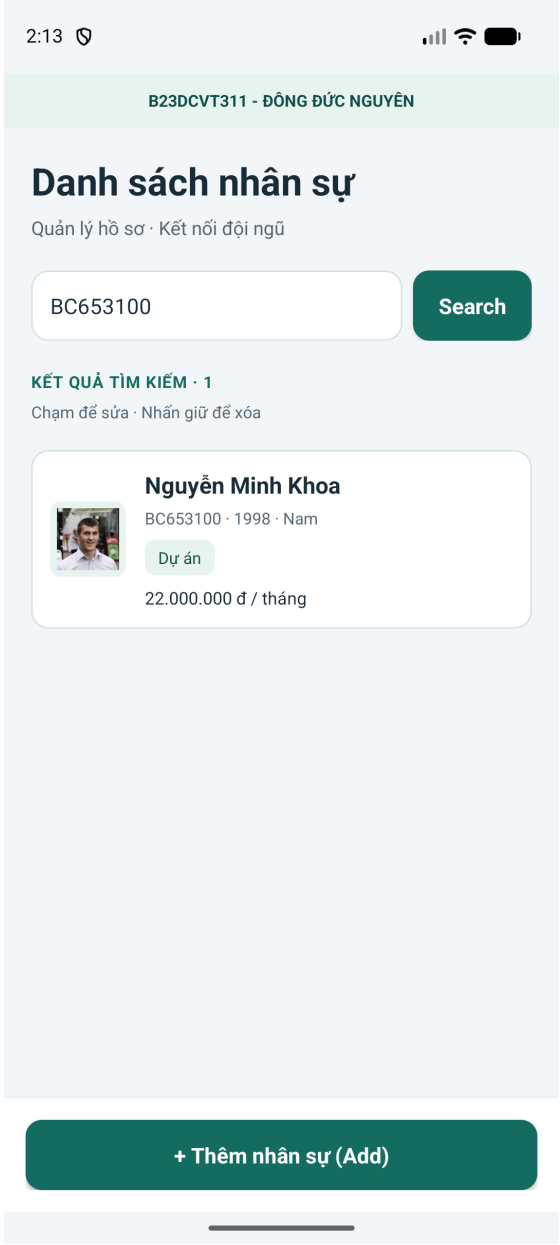
11 / CẬP NHẬT HỒ SƠ

# Update lưu phòng ban và mức lương mới

Mở lại Detail; đổi phòng ban sang Dự án và mức lương thành 22.000.000 VNĐ.




Hình 14. Thông tin công việc đã sửa, chưa lưu.




Hình 15. Main hiển thị phòng Dự án và lương mới.

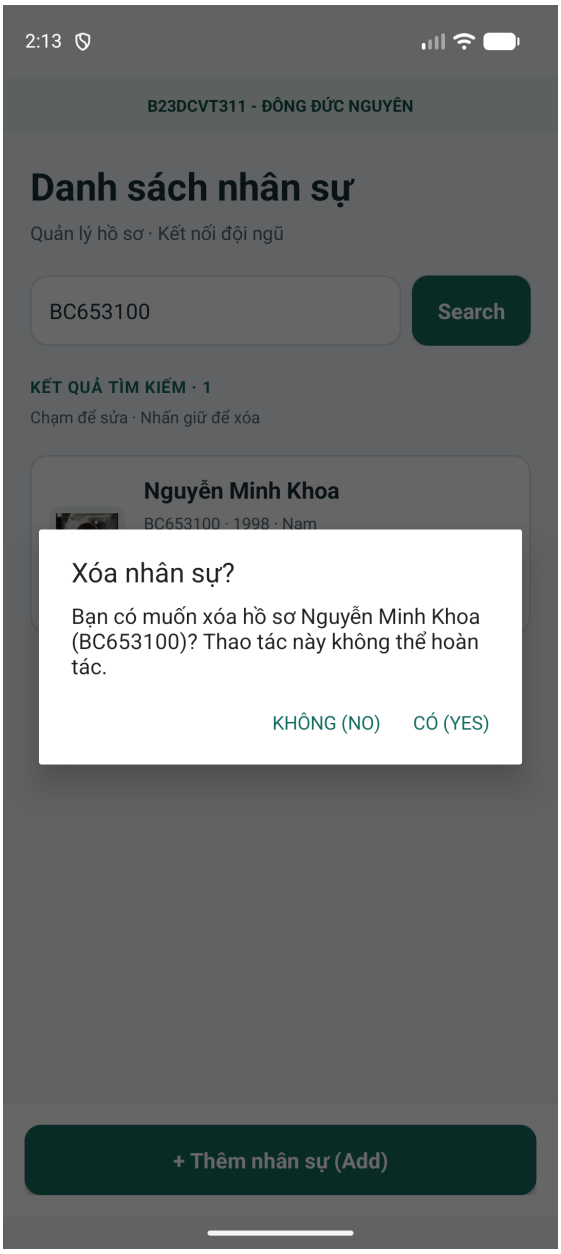
## KẾT QUẢ QUAN SÁT

Update giữ nguyên _id của hồ sơ, lưu đúng phòng Dự án và mức lương 22.000.000 VNĐ. Main tải lại dữ liệu sau khi lưu thành công.

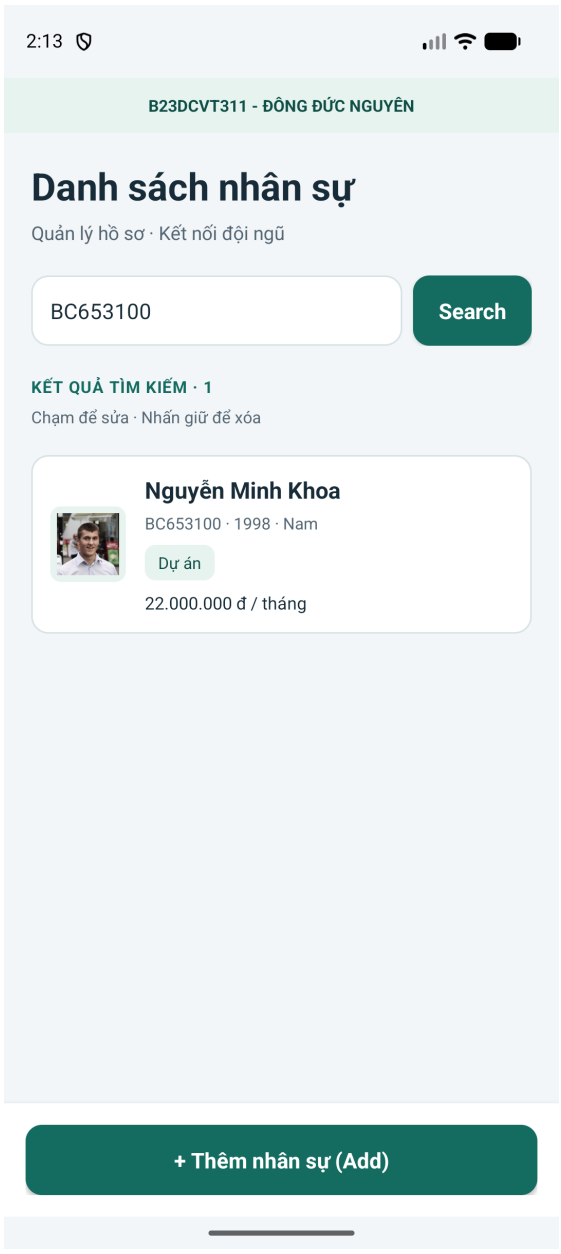
12 / XÁC NHẬN XÓA

# Nhấn giữ và chọn Không (No)

Nhấn giữ dòng nhân sự để mở hộp thoại xác nhận; chọn Không (No).




Hình 16. Hộp thoại xác nhận có tên và mã hồ sơ.




Hình 17. Chọn No: nhân sự vẫn còn trong danh sách.

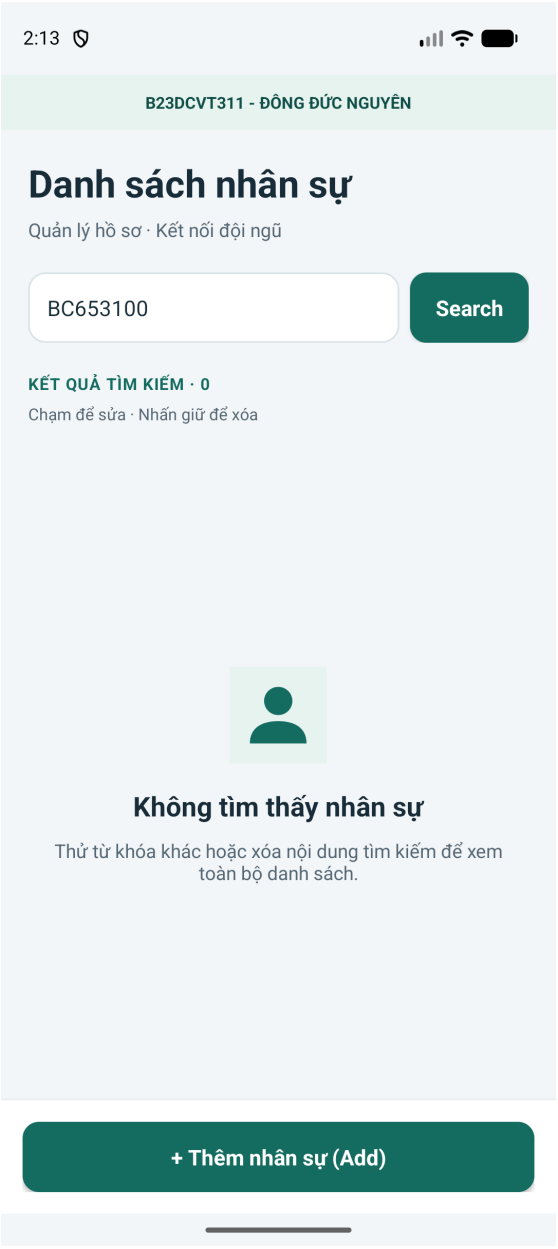
**KẾT QUẢ QUAN SÁT**

Nhấn giữ không mở nhầm Detail. Chọn No chỉ đóng hộp thoại; bản ghi và mọi thông tin của nhân sự vẫn được giữ nguyên.

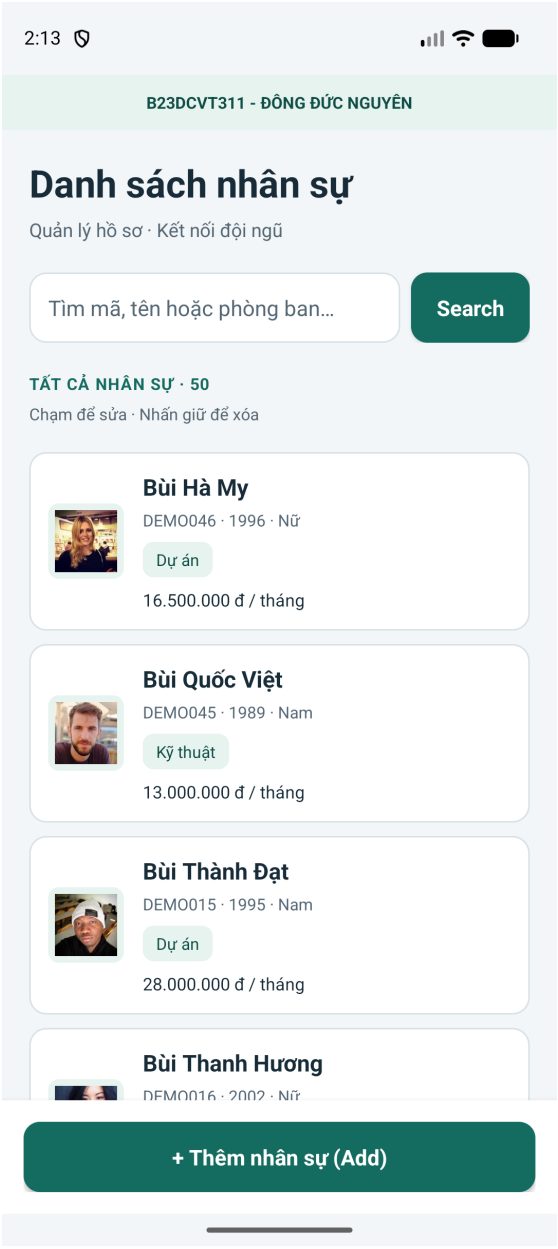
13 / XÓA VÀ HOÀN TẤT

# Chọn Có (Yes) để xóa hồ sơ

Nhấn giữ lại hồ sơ tạm, chọn Có (Yes); sau đó xóa từ khóa tìm kiếm.




Hình 18. Sau Yes, tìm theo mã vừa xóa không có kết quả.




Hình 19. Trở về danh sách 50 nhân sự mẫu ban đầu.

## KẾT QUẢ QUAN SÁT

SQLite không còn hồ sơ tạm. Danh sách trở về 50 nhân sự, chứng minh chuỗi thao tác báo cáo không làm mất dữ liệu mẫu có sẵn.

14 / TỔNG KẾT

# Kết quả kiểm tra và nhận xét

Các ảnh trong báo cáo được chụp trực tiếp từ ứng dụng trên máy ảo Pixel 9.

| Nội dung kiểm tra | Kết quả quan sát | Đánh giá |
|---|---|---|
| Danh sách / Search | 50 hồ sơ; tìm phòng ban không dấu; có trạng thái không kết quả. | Đạt |
| Refresh / dữ liệu bắt buộc | Xóa ô nhập; chặn lưu form thiếu mã nhân sự. | Đạt |
| Chọn ảnh / Add | Ảnh đọc được; bản ghi tăng lên 51; tìm đúng hồ sơ mới. | Đạt |
| Detail / Cancel | Mở đủ thông tin; hủy sửa giữ nguyên họ tên cũ. | Đạt |
| Update | Lưu phòng Dự án và lương 22.000.000 VNĐ. | Đạt |
| Long click / No / Yes | No giữ bản ghi; Yes xóa đúng hồ sơ; còn 50 nhân sự. | Đạt |

## Nhận xét

Ứng dụng đáp ứng luồng quản lý nhân sự theo yêu cầu: giao diện rõ ràng, tách lớp dữ liệu và giao diện, có xác nhận xóa và kiểm tra đầu vào. Bản build đã qua kiểm tra nền gồm 3 unit test và 5 Android test; kịch bản chụp báo cáo cũng hoàn thành thành công.

## Phạm vi và hướng phát triển

Minh chứng chạy trên một máy ảo; chưa thay thế kiểm thử nhiều thiết bị. Có thể mở rộng phân trang, chuyển truy vấn SQLite sang luồng IO, xuất dữ liệu và bổ sung sao lưu. Ảnh mẫu được lưu cục bộ; ảnh người dùng chọn phụ thuộc quyền đọc và tệp nguồn còn tồn tại.

### Nguồn tham khảo và ảnh minh họa

1. Android Developers, SQLiteOpenHelper.
2. Ảnh chân dung: Random User - Copyright Notice. Tên và thông tin nhân sự là dữ liệu giả lập.
3. Mã nguồn dự án MB3; nhật ký chạy: docs/report-capture-result.txt.